

로그 RCA를 위한 KV 압축 — 슬림 기획서

선행연구는 평가를 어떻게 설계하고 수치를 정했는가, 그리고 우리는 어떻게 하는가

평가 설계 중심 정리본 · 2026 · LogPress

기존 KV 압축기(SnapKV·H2O 등)를 로그 RCA로 처음 돌려서 원인 라인을 얼마나 보존하는지 측정하고, 부족한 지점(고압축·하드케이스)을 로그 고유 신호로 개선한다.

이 문서는 세 질문을 척추로 한다: ① 선행연구는 평가설계를 어떻게 세웠나 · ② 수치를 어떻게 정했나 · ③ 안 정했다면 대신 무엇을 했나. 두 편(LogSieve, KVDiagnosis)을 이 세 칸에 넣어 우리 평가 설계의 근거를 세운다.

0. 비교 구도 — 되는 비교와 안 되는 비교

모든 혼란의 뿌리는 "무엇과 비교할 것인가"였다. 기준은 하나다 — 같은 무대(층위)에 세울 수 있는가.

구분	누구	왜 그 자리인가
baseline 같은 층위 숫자로 정면승부	SnapKV · H2O · random KV · 레벨 필터	목적은 달라도 KV 층위로 같아서 같은 축(압축률 – 보존율)에 나란히 세워짐. 우리가 숫자로 이겨야 할 상대.
related work 다른 층위	LogSieve · LoFI (RCA-aware 축소) LogReducer · LogShrink · LogZip (로스리스)	목적은 같으나(원인 줄 남기기) 개입 층위가 다름 . 로스리스는 목표 자체가 다름(바이트 무손실). 이기는 대상이 아니라 빈칸을 보여주는 배경.

층위 차이 한 문장

LogSieve: 이 줄을 LLM에게 보여줄까 말까를 텍스트 단계에서 정함(입력 필터, 비가역, head/layer 구분 불가). **LogPress**: 이 줄의 KV를 캐시에 남길까 말까를 모델 내부에서 정함. 같은 줄도 head별로 다르게 유지 가능 → 회색지대·공간에 묻힌 문맥을 다룰 수 있음.

단, LogSieve도 2층에서는 비교 가능. 1층(압축률-보존율)은 축이 달라 나란히 못 놓지만, 2층(최종 RCA 정답률)은 "같은 토큰 예산에서 RCA를 얼마나 지키나"라는 공통 결승선이 있어 붙일 수 있다.

① 선행연구는 평가설계를 어떻게 세웠나

두 계열이 서로 다른 것을 가르쳐준다. **LogSieve = 로그를 어떻게 다룰지**(정답지·데이터 관행), **KVDiagnosis = KV 압축을 어떻게 평가할지**(평가 통계 관행). 둘은 겹치지도 않고 상보적이다.

LogSieve — 무엇과 비교했나, 그리고 정답지를 어떻게 만들었나

LogSieve는 경쟁 압축기와 붙지 않았다(그런 경쟁자가 거의 없어서). 대신 "원본 / 자기 / 무작위 / 구조기반"이라는 축소 강도·방식 스펙트럼과 비교했다.

- rate-matched random 대조군**: LogSieve와 똑같이 42%를 지우되 무작위로 지운 자작 대조군. "무엇을 지우냐"만 격리해 우연·잉여 가능성을 배제. → 우리 random KV baseline이 이 역할을 이식.
- 다운스트림 과제로 검증**: 축소 자체가 아니라 "축소한 것을 LLM에 넣었을 때 RCA가 유지되나"로 평가. → 우리 2층과 동일. 정답지 라벨 종류에 덜 오염되는 안전장치.
- 정답지를 "관련성" 기준으로 재정의**: 기존 이상 라벨을 그대로 쓰지 않고 라인마다 RCA 관련성(관련/무관)을 직접 주석해 그라운드 트루스를 만들. → "이상 라벨이라 baseline이 유리했던 것 아니냐"는 우리 고민의 정확한 해법.

KVDiagnosis — KV 압축 평가의 통계 골격

관행	내용 · 우리에게 주는 것
matched-pair (FullCache 짝대조)	압축 실행마다 압축 안 한 원본(FullCache)을 짝으로 같이 돌리고 model·prompt·tokenizer·decoder·scorer를 고정. "압축본 96%"만 보면 좋은지 나쁜지 모르지만, 짝지으면 압축 때문에 생긴 손실만 격리 . → 우리 2층 McNemar의 정당화.
C→W 행 선택 ("잘 안 됐다"의 정의)	FullCache-correct / compressed-wrong — 원본은 맞았는데 압축이 틀린 행. "점수 낮음"이 아니라 "압축만 아니었으면 맞았을 걸 압축이 죽였다". 우리로 치면 "FullCache는 원인 보존했는데 SnapKV는 놓친 줄" = 우리가 이겨야 할 지점. 선택은 모든 실행이 끝난 뒤에 해사 후 선택 편향 차단.
명시적 분모 + 결측(N/A)	젤 수 없는 것은 0으로 안 채우고 not_applicable로 따로 뺌(EAR은 정의되는 56.2% 행에서만 계산). → 우리 non_error=0은 "보존율 0%"가 아니라 "N/A(해당 원인 없음)" 로 기록해야 정직.
C→C 성공 대조	실패만 보면 편향. 둘 다 맞힌 행을 대조로 같이 봐 "실패 케이스가 성공과 진짜 다른가"를 확인. → "왜 우리가 이기는지"의 설명 근거.
구현 감사(audit)	PyramidKV를 구현 결함으로 제외(SnapKV와 같은 경로로 잘못 돌아 7,800쌍 동일값). → baseline을 kvpress에서 가져와도 "진짜 그 방법으로 도는지" 검증. week7의 "소수점 똑같음"도 이 감사로 점검할 신호 .

② 선행연구는 수치를 어떻게 정했나

LogSieve — 사후 제시, 게다가 지표가 방법에 종속

목표 수치를 미리 세우지 않았다. fidelity(Cosine·GPTScore ≈ 0.93 등)를 결과로 제시했을 뿐이다. 더 중요한 건 **그 지표를 우리가 못 가져온다**는 점: LogSieve의 fidelity는 "축소된 텍스트를 LLM에 넣었을 때 답이 원본과 얼마나 비슷한가"라, KV를 압축하는 우리에게겐 텔 텍스트 자체가 없다. 목적은 공유하지만 지표가 방법에 묶여 있어 복불 불가.

KVDiagnosis — 목표 달성치를 안 세우고, 판정 임계값을 "사전선언"

이 논문에서 손으로 고른 숫자는 목표치가 아니라 **진단 임계값**뿐이고, 전부 결과에 맞춘(fit) 게 아니라 미리 선언(predeclared)한 값이다.

사전선언한 값	의미
ECov 커버리지 임계값 $\tau = 0.5$	원인 증거 스펙의 50% 이상 보존 시 "커버됨". 저커버리지 40.3% / 부분 22.9% 비율이 이 컷오프로 걸림 — 손으로 정한 값.
Δ -NLL 강한 드리프트 = 1.0	likelihood가 그만큼 흔들리면 "강한 드리프트".
evidence-attention 4× 부스트	개입(복구) 실험의 크기.
압축률 격자 75 / 50 / 25%	KV 유지 비율을 결과 전에 고정. 우리 50~90%에 대응.
explicit N/A 2,600개	안 되는 건 0이 아니라 N/A로 장부에 남김.

규모: 2,600 FullCache 대조 + 62,400 method-setting 레코드(59,800 supported + 2,600 N/A), 12,520 C→W 행, 8개 압축기. 성공 기준도 "몇 % 달성"이 아니라 **판별력**이었다 — C→C 대비 10개 진단 전부가 실패/성공을 분리(stratified AUROC 0.684–0.871), 저-EAR 실패 4× 부스트로 29.2% 복구 vs sham 6.3%.

핵심 발견

조사한 압축 논문 중 "**성공 = 목표치 X% 이상**"을 **결과 전에 세운 사례는 사실상 없다**. KVDiagnosis는 목표치 대신 **판정 임계값을 미리 선언**했고, RULER 정도만 외부 앵커(Llama-2-7B의 4K=85.6%) 기반 임계값을 쓴다. 즉 우리가 "판정 규칙은 미리, 값은 나중"으로 가는 것은 관행에 어긋나지 않으며, 제대로 하면 오히려 앞선다.

③ 안 정했다면 대신 무엇을 — 우리 방식

대원칙

값은 진행하면서, 규칙은 진행 전에. baseline·우리 압축기의 실제 보존율(값)은 돌려봐야 나온다. 하지만 "얼마나 이기면 성공이라 부를지"(판정 규칙)는 결과 보기 전에 못 박는다 — 그래야 "결과에 목표 맞추기(어뷰징)"를 막는다.

우리 판정 규칙 (결과 전에 사전선언)

1층 목표 — 실무 근거형

"압축률 **90%**, 하드케이스(비-ERROR · 앞쪽 쏠림) 층에서, **최고 baseline이 원인 0개 보존한 케이스를 우리 압축기가 1개 이상 보존한 비율이 유의하게 높다** (McNemar $p<0.05$ + 부트스트랩 95% CI 하한 > 0)."

- **실무 근거** ✓ — "원인 0개 = 진단 불가"라는 RCA 현실에 직접 걸림
- **자의적 % 없음** ✓ — 절대 %가 아니라 개수/케이스 기반
- **통계 검정** ✓ — McNemar + 부트스트랩
- **사전등록 가능** ✓ — baseline 숫자를 몰라도 성립(상대식)

사전선언할 임계값 (KVDiagnosis τ 방식 차용)

- **보존 판정 컷오프**: 원인 라인 토큰이 몇 % 이상 살면 "보존 1"로 볼지(기본 rule=any). fit 아니라 declare.
- **하드케이스 정의**: "앞쪽 쏠림"의 위치 범위를 미리 고정(③ 문치 내부 좌표계 기준).
- **압축률 격자**: 50 / 70 / 80 / 90%를 미리 고정, 주 판정점 = 80~90%.

N/A 처리 · 보조 성공기준

- **N/A를 장부에 남긴다**: non_error=0(예: chunk13)은 "보존율 0%"가 아니라 **N/A(해당 원인 없음)**로 기록. (KVDiagnosis가 2,600개를 그렇게 함.)
- **보조 성공기준(선택)**: 절대치 없이도 성립하는 판별력 지표 — "우리 로그 신호가 원인/비원인을 AUROC로 유의하게 가른다"를 함께 보고.

4. 계산법 — 메트릭 정의

- **그라운드 트루스(정답지 G)**: "진짜 원인 줄이 무엇인지" 미리 확정된 집합. *책점에만* 쓰고 압축 시점엔 안 봄(라벨 누수 금지). Thunderbird의 이상(anomaly) 라벨로 시작하되, 이는 "원인" 라벨이 아니므로 Nezha(원인 라벨)로 교차검증.
- **1층 보존율**: 고정 압축률 c에서 **G 중 압축 후 살아남은 비율**. 이진(rule=any: 원인 줄 토큰 하나라도 살면 1). 반드시 (a) 전체 (b) 비-ERROR (c) 앞쪽 쏠림 층으로 나눠 보고.
- **판정 통계**: 같은 케이스에서 baseline vs 우리 승패를 McNemar로, 차이 신뢰구간을 부트스트랩(95% CI 하한 > 0)으로.
- **2층**: 같은 압축률에서 압축 방식만 바꿔 **제약형 RCA(K지선다)** 정답률 delta. 자유서술은 오픈 모델 정답률이 바닥이라 제약형으로 floor를 띄움.

5. novelty 방어 — 예상 공격과 답

예상 공격	우리 방어
"로그로 평가된 적 없다"는 걸 어떻게 아나? 없음의 증명 아닌가	"존재하지 않는다" 대신 검증 가능한 형태로 : "KV 압축의 표준 평가셋(LongBench·RULER·NIAH)에 로그 RCA 테스트가 없다"(구성표로 확인되는 사실).
도메인이 다르다고 성능이 다르다는 보장 있나 (so-what)	부재가 아니라 메커니즘 으로: 어텐션·최근성·위치라는 대리지표가 (a) 평범한 INFO 원인엔 안 몰리고 (b) 질문 근처가 아닌 중간 원인을 못 잡고 (c) 최근/시작 규칙으로 중간 원인을 버림. 단, week7에서 저압축·ERROR 원인은 어텐션도 잘하므로 주장은 고압축·하드케이스로 한정 .
"처음 해봤다"는 application novelty 아닌가	novelty의 주어를 로그가 아니라 방법으로: " 어텐션 대신 로그 고유 구조 신호로 KV를 고르는 첫 방법 " = 중요도 산정 방식 자체를 바꾼 method novelty.
결국 "로그로 평가 안 됐다"는 어떻게 메우나	우리가 그 평가가 된다 . 기존 KV 압축기를 로그로 처음 돌려 격차를 숫자로 박음. 말싸움은 반박당해도 실측은 반박 못 함. (조건: 정답지 + 비-ERROR 원인 데이터.)

6. 지금 할 일 · 필요 데이터

1순위 — 데이터가 목표를 따라온다

위 판정(“baseline 0개 → 우리 1개+”)이 성립하려면 **비-ERROR 원인이 든 정답지**가 반드시 있어야 한다. week7 chunk13은 non_error=0이라 이 주장을 증명할 케이스가 아예 없다. → **비-ERROR 원인 뭉치(또는 Nezha 최소 슬라이스) 확보가 최우선**.

- baseline 재현(kvpress) + **구현 감사**(“소수점 똑같은” 점검) + random KV·레벨필터 추가
- token→line 매핑 완성(하이브리드의 핵심, scorer가 기다리는 인계물)
- 판정 규칙·임계값 사전등록 문서 동결(결과 전 타임스탬프)

출처 · 검증 주의. LogSieve(arXiv:2601.20148, MSR 2026) · KVDiagnosis(arXiv:2608.09412, GitHub ChosenQC/KVDiagnosis). KVDiagnosis의 수치($\tau=0.5$, $\Delta\text{-NLL}=1.0$, $4\times$, AUROC 0.684–0.871, 59,800/12,520, 75·50·25%)는 검색 요약 기반이므로 인용·차용 전 **원문 및 GitHub record format으로 직접 재확인** 필요. baseline 수치는 논문값을 인용하지 말고 kvpress로 **직접 재현한 값만** 사용.